DESIGN & ANALYSIS OF ALGORITHMS
PROJECT REPORT SUBMISSION FORM EVIDENCE SHEET

Assignment (1-6): 1	Course / section: DAA / AIML Panthers	Submitted on: 10-10-2026
Project / framework title: Algorithm 2035: Adaptive Algorithm Selection Framework for Smart Cities		
Team: AlgoNex Koneri Mithul — 25WU0102129 L Bilvesh Datta — 25WU0102135 K. Sandeep — 25WU0102121 K Sharva teja — 25WU0102128 A Rohan reddy — 25WU0102329		
Code / repository URL: https://github.com/konerimithul-lgtm/algorithm-2035-smart-cities	Report / demo link: Not provided yet	

PURPOSE Demonstrate how you model a real problem, justify a syllabus algorithm, analyze its cost, implement it, test it, and adapt the choice when conditions change.

1 PROBLEM ABSTRACTION & MODELLING LO1 · model real world scenarios

Scenario / decision to solve: Choose and apply algorithms for smart-city network, routing, budget-allocation, and search/optimization tasks.	
Entities / inputs; size parameter(s): Zones/vertices V, roads/edges E, project benefits and costs, capacity W, search size N.	Objective + constraints: Minimize network cost; compute shortest paths; maximize project benefit under budget; solve constraint-search examples.
Representation: [✓] graph [✓] optimization [✓] state/search [✓] DP [✓] data structure Other: weighted graphs	

2 ALGORITHM CHOICE & FRAMEWORK DESIGN LO2 · choose & compare

Implemented DAA syllabus techniques: Prim, Kruskal, Floyd–Warshall, Fractional and 0/1 Knapsack, N-Queens and TSP backtracking, B-Tree and Red-Black Tree.
Alternative compared: Prim vs Kruskal; Fractional vs 0/1 Knapsack; B-Tree vs Red-Black Tree. Selection rule: graph density <0.35 selects Kruskal; otherwise Prim.
Workflow: scenario → model → choose → execute → measure → explain. Architecture diagram: include in report; page number to be added.

3 COMPLEXITY & CORRECTNESS LO3 · reason formally

Major technique / operation	Expected time T(n)	Space S(n)	Assumption / correctness argument
Kruskal MST	O(E log E)	O(V + E)	Sort edges and use union-find; selects safe non-cycle edges.
Floyd–Warshall	$O(V^3)$	$O(V^2)$	Relaxes all intermediate vertices; valid for weighted graphs without negative-cycle issues.
Complexity status: [✓] polynomial [✓] pseudo-polynomial (0/1 Knapsack) [✓] NP-Hard optimization (TSP) [✓] NP-Complete decision (TSP decision)			

4 IMPLEMENTATION & VALIDATION LO4 · build & verify

Language / tools and implemented modules: Python; VS Code; main.py; data_structures.py; unittest tests; GitHub.
Correctness tests: [✓] normal [✓] boundary [✓] large [✓] difficult (confirm coverage) Tests passed: 8 / 8 Code proof: test file / terminal output; page to be added.

5 EXPERIMENTS & ADAPTATION LO5 · measure & adapt

Input / condition	Size n	Algorithm / structure	Runtime / ops / memory	Result / observation
Baseline: sparse random graph	V=20, E=59	Prim vs Kruskal	0.113 ms vs 0.051 ms	Same MST cost: 125; Kruskal faster in this run.
Changed: larger sparse graph	V=100, E=299	Prim vs Kruskal	1.153 ms vs 0.219 ms	Same MST cost: 679; timings vary by machine/run.
Decision/behaviour changed because graph density and input size affect the heuristic and measured runtime. Evidence: MST benchmark output; add report page number.				

6 EXPLANATION, LIMITS & DEMONSTRATION LO5 · interpret & communicate

Main finding and limitation: The framework demonstrates multiple algorithm choices and comparisons. Backtracking TSP scales poorly as the number of cities grows.
Theory versus observation: Big-O describes growth; measured times also depend on implementation, hardware, input shape and timing noise.
NP-Hard / NP-Complete: TSP optimization is NP-Hard; its decision version is NP-Complete. Exhaustive backtracking is impractical for large instances.

SUBMISSION CHECK [ ] integrated framework [ ] working code [ ] alternative compared [ ] time/space analysis [ ] 4 test families [ ] adapted decision [ ] report + demo

ASSESSOR: Modelling __/5 | Selection __/5 | Analysis __/5 | Code __/5 | Experiments __/5 | Demo __/5 TOTAL __/30

Faculty feedback: _____

Before submission: replace the report/demo placeholder and page references after preparing the final report/demo. Benchmark times are sample results from one run and may vary.